The goal of the type systems in Ruby is to allow for as many of the benefits of typed languages as we can get without giving up the flexibility that makes Ruby, Ruby.

Official Ruby Typing with RBS

The official Ruby typing system shipped with Ruby 3.0 is called RBS (which is short for Ruby Signature). With RBS, you create a separate file that contains type signature information for all or part of you code.

Writing RBS

To take a look at how RBS works, we'll use the gem we created in Chapter 15, Ruby Gems, on page 251 and augment it with RBS typing. If you look at the Aaagmnr gem code, you'll see that it contains a directory named sig that we didn't talk much about. That directory is where you are supposed put the type information, and right now it contains one file:

gems/aaagmnr/sig/aaagmnr.rbs

```
module Aaagmnr
  VERSION: String
  # See the writing guide of rbs: https://github.com/ruby/rbs#guides
end
```

The only thing that file tells us is that the module Aaagmnr has a VERSON constant, which is a String. True enough, but not very useful.

Here's what an RBS file for the entire gem looks like:

typed_ruby/aaagmnr/sig/aaagmnr.rbs

```
module Aaagmnr
  class Finder
    @signatures: Hash[String, Array[String]]
    def self.from_file: (String file_name) -> Finder
    def initialize: (Array[String] dictionary_words) -> void
    def lookup: (String word) -> Array[String]
    def signature_of: (String word) -> String
  end

  class Options
    attr_reader dictionary: Array[String]
    attr_reader words_to_find: Array[String]

    def initialize: (Array[String] argv) -> void
    private def parse: (Array[String] argv) -> void
  end

  class Runner
    @options: Array[String]
    def initialize: (Array[String] argv) -> void
    def run: () -> void
  end

  VERSION: String
end
```

The goal here is to describe the expected types of all the modules, constants, and methods in the gem. The syntax is meant to be similar enough to Ruby to be readable, while still providing for type information.